

⚡ Quick AI Read

🧩 Quiz

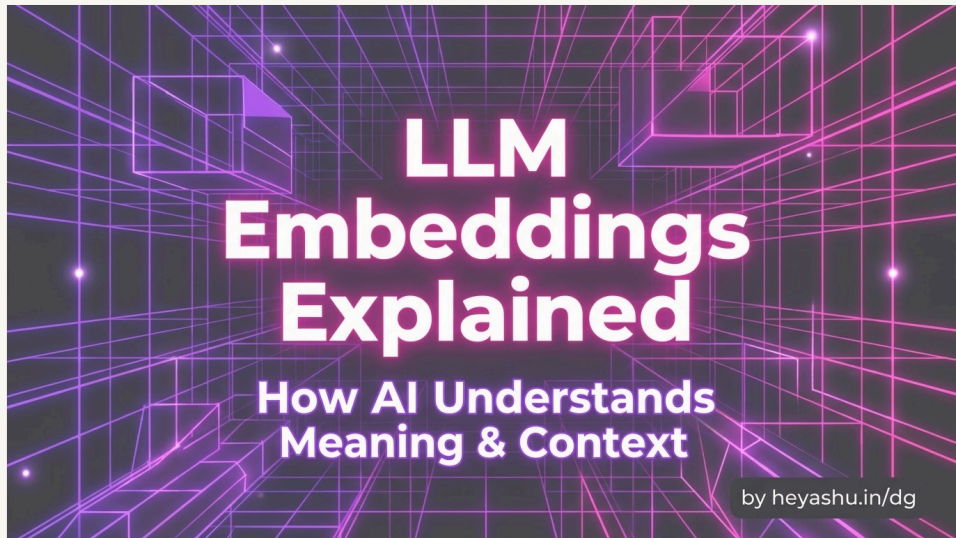
💬 Q&A

LLM Embeddings Explained: How AI Understands Meaning & Context



By Ashutosh Anand Tiwari

Published Aug 24, 2026 · 15 min read



Let's dig deep into LLM language, how AI understands similar words in different ways. Like I say, **I ate apple at Apple Store**, how does AI know the first apple is an eatable thing and the second Apple is the company name? So, it knows that. This is the main mystery we are going to crack.

Sometimes we only get confused, so how does a machine understand? How does NLP solve this issue? We already know about tokenization and token IDs and all, so let's go next.

Or whether that token makes sense or is just a random number. Like if **Dog** is represented with ``8123``, does that make sense? **Grapes** are ``8521``; are those two related? But actually, no. The numbers are near, so the answer is no, they are just dummy token ID numbers associated with those words/tokens.

So token ID doesn't contain meaning. Remember this, and no two IDs are related to each other. Then how?

Vectorization

It's the process of converting information into numerical vectors. It's like an array of numbers.

“Vectorization in large language models is the process of turning words, sentences, or images into long lists of numbers so that a computer can understand their meanings and relationships.”

So any information, text, document, words, images, anything can be converted into an array of numbers. But this is happening because computers can perform mathematical operations on top of it.

So suppose you need to rank fruits based on three properties: sweetness, size, and crunchiness. So for Apple, it will be ``[.7, .4, .8]``, for Banana ``[.8, .6, .2]``, and for Carrot ``[.2, .5, .9]``. So this is just an example based on those three characteristics.

So, on certain dimensions, these numbers are converted into vectors. Like Apple here and other fruits are ranked on dimensions like sweetness, size, and crunchiness. So like this, other words can also have multiple dimensions, and vectorization happens based on that.

So this is done by embedding models, and the process is called vectorization. The embedding is a large array of numerical numbers. All companies have their own embedding models.

Modern embedding models use thousands of dimensions. Those dimensions are learned from the data, the vast data they got trained on. So Apple Store and apple fruits are identified easily.

So Apple with sweetness and crunchiness was talked about multiple times, and Apple with buying Mac devices and iPhone was discussed and represented multiple times, so dimensions are created based on all that data.

We will discuss more on it later about embeddings, but for now, we just discussed overall how it happens.

Embedding

An embedding in an LLM is a way to turn words, sentences, or pieces of text into a list of numbers that capture their actual meaning. So, embedding is a learned numerical representation of an item that captures useful relationships with other items.

So, for example:

King - `[.81, .32, .52]`

Queen - `[.79, .36, .48]`

Banana - `[.24, .91, .11]`

So, on this embedding, you can see King and Queen are relatable, but not with Banana. So, if we have thousands of dimensions, we can have relationships mathematically possible.

And this embedding happens based on the data we have and the relationships. Embeddings show relationships. So, associated words' embedding values will be together. Just remember it.

And this happens naturally based on data, based on some algorithms, and lots of GPUs are required to do this continuously on upcoming fed data. The model is not given these values by a human, that is clear now.

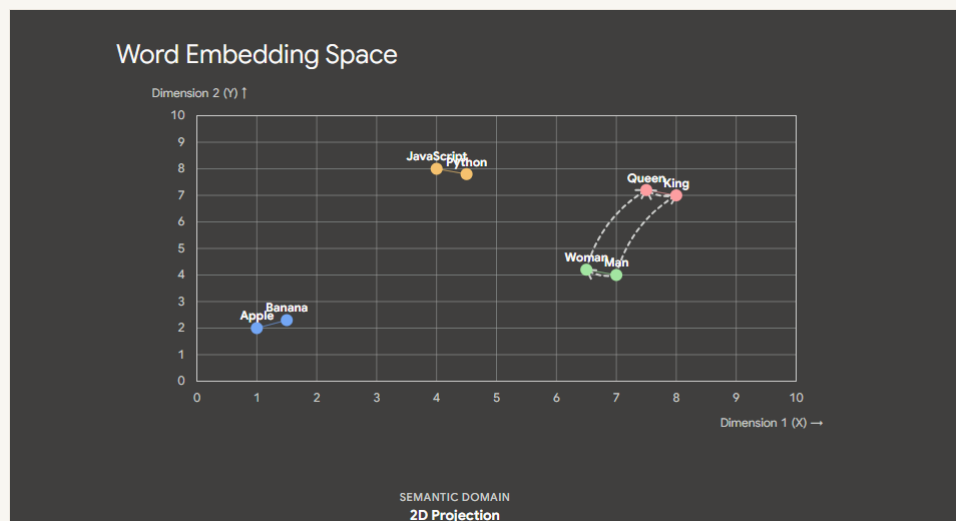
During training, the embedding values are learned. So, in the last word, it develops numerical relationships. Embedding is not the meaning of the words.

Embedding as coordinates

Let's try to visualize it in a 2D plot with only 2 dimensions, with this example.

These coordinates illustrate semantic clustering and geometric vector relationships in a 2D embedding space:

- **Fruits:** **Apple** (1, 2) and **Banana** (1.5, 2.3) cluster in the lower-left quadrant.
- **Programming Languages:** **JavaScript** (4, 8) and **Python** (4.5, 7.8) group near the upper-center.
- **Human Roles:** **Man** (7, 4) and **Woman** (6.5, 4.2) sit centrally on the right.
- **Royalty:** **King** (8, 7) and **Queen** (7.5, 7.2) sit in the upper-right.



Notes: More dimensions do not mean more intelligence. Higher dimensions can capture complex patterns, but they also require more storage and computation. And with each training, these numbers are adjusted.

Semantic similarity

Semantic similarity means measuring how close two pieces of text are in the same meaning. I mean, different kinds of statements but have the same meaning. Like Query A: How do I center a div? Query 2: How can I align an HTML element in the middle of its parent? So, those both have similar meaning but different words.

In easy words, Semantic similarity in large language models is **a way for computers to check if two sentences mean the same thing, even if they use completely different words.**

But making this possible was not possible and was complex because of word combinations. But semantic similarities can be related if we use embeddings, and this is only possible because of embeddings. And note one thing: embedding of words is different from embedding of tokens, so we will discuss all of this later.

Embedding is the core thing for AI models. Learn from a research paper and learn about the embedding thing. It would be very fun to learn.

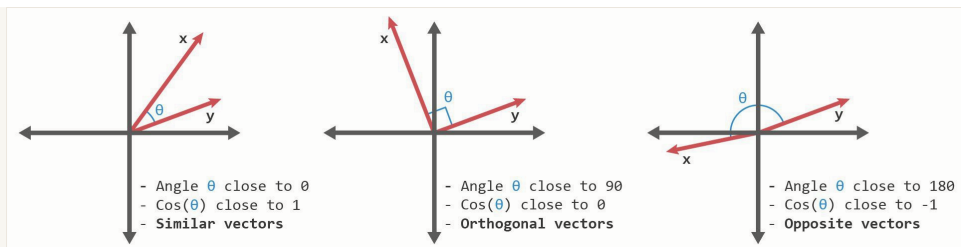
Embedding allows a system to compare broader meaning rather than only matching identical words.

COSine Similarities,

The process of achieving semantic similarity is called **cosine similarity**.

Cosine similarity **measures how close the meanings of two pieces of text are by looking at the angle between their arrow representations (vectors).**

Learn here: <https://www.ibm.com/think/topics/cosine-similarity>



If two vectors point in a similar direction, the represented concepts can be similar. If they point in unrelated directions, the concepts may be less similar.

Read the above-mentioned blog, you will learn more if you want to go deeper into the maths. I'm not going too deep into that, sorry ☹. I'm just thinking there is a factor that calculates vector direction, and based on that, semantic similarities get identified.

You can read here too: <https://www.learndataasci.com/glossary/cosine-similarity/>

Two Similarity Does Not Understand Truth

Two false statements can be semantically similar.

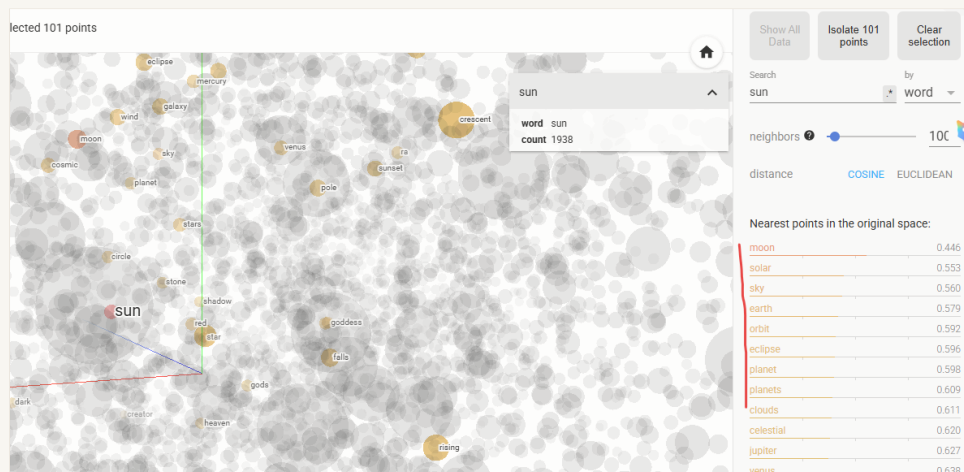
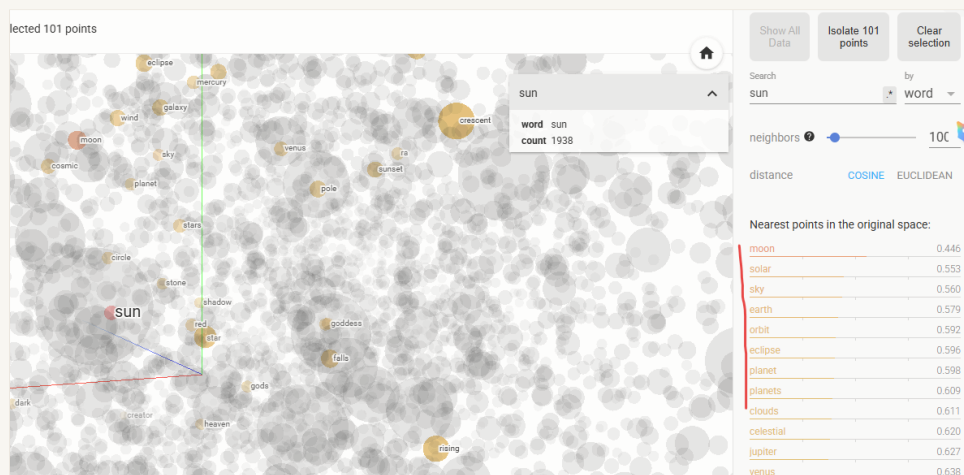
Two harmful instructions can be semantically similar.

Two texts can be close in topic but disagree completely.

“Embeddings capture relationships. They do not independently verify facts, intent, quality, or safety. Remember it.”

If you want to visualize the embedding, go to this website: <https://projector.tensorflow.org/>

Here I'm pasting two screenshots. See the relation in the embedding projector of **sun** and **JavaScript**, and its nearby embedded words.



Token Embedding

So never get confused: a token can also have an embedding, and a word can also have an embedding. So token ID has no meaning, as we know, but the token IDs can have embeddings of an array, and that embedding of that token shows the relation between words and between tokens we can see.

So token IDs are meaningless, and embeddings are meaningful vectors.

So token embedding relates a meaningless token ID with a dense numerical vector. That vector becomes the model's initial representation of the token, because the representation may later change as the model processes the context.

“Token embedding turns a piece of text into a list of numbers that a computer can use to understand meaning.”

So a sentence like ``I love JavaScript`` is converted into tokens and token IDs. The token IDs are converted into embeddings like an array, and they relate to some other embeddings, and this forms a relationship.

But remember, it can change as the model processes the context. I mean, the way that token and word embeddings are used in a sentence with other words or tokens can change based on the context.

Positional embedding

So let's take an example where we say **Dog bites Man, Man bites Dog**. So the embedding for **bites** would also be the same for these two, but the context output will be different. How? And this all happens based on positional embeddings. So the order of the embedding, or you can say that the token or word position, also matters.

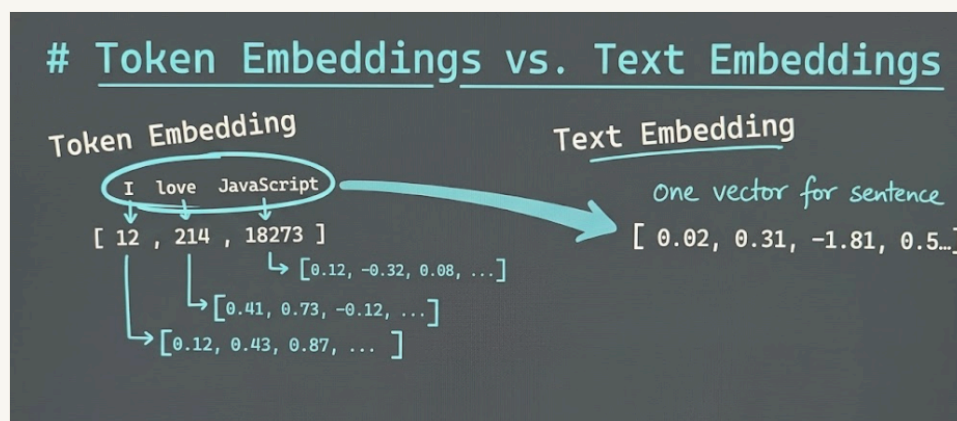
“ Positional embedding is a technique used by large language models (LLMs) to give words a sense of order and sequence in a sentence.”

The model also needs information about the order or position before answering the query. Models therefore incorporate positional information using architecture-specific techniques.

The embedding tells the model which token is present. Positional information helps tell the model where that token appears, and the model can identify both the token and its order.

Token vs Text Embedding

Token embedding helps a language model to process a sequence internally. Text embeddings are often produced specifically for tasks such as search, retrieval, clustering, recommendation, classification, duplicate detection, etc. So when the task is about the order and all of the tokens, then token embedding matters, and when we search, the complete text embedding comes.



“ The main difference is scope: Token Embeddings turn individual pieces of words into numbers to help a model read text, while Text Embeddings (often called sentence or document embeddings) turn an entire sentence, paragraph, or document into a single mathematical summary to capture its overall meaning.”

Think of it like looking at a book: a **Token Embedding** is a definition for a single syllable or word, while a **Text Embedding** is the summary printed on the back cover of the book.

And with the previous example, we understood that contextual and positional embeddings help words form relationships. A static embedding will not work overall.

Polysemy means a **single word has many different meanings depending on the sentence**.

How LLMs handle context

Now, from a long time, we are using word context, like order changes the context. With training, after context values can be changed, but let's understand how a model handles the context.

For example:

Java is a language.

Java is an island in Indonesia.

In a Transformer-based language model, the token first receives an initial token embedding, then the model processes it together with the surrounding tokens.

So suppose for our previous example, **Java** in both statements can have the same embedding `[.91]`. Then the model processes it together with the surrounding tokens. Through multiple layers, the representation becomes contextualized, and it all happens using Transformer architecture. We will discuss more about this later.

This initial word **Java** can become different in the second statement: **Java is an island in Indonesia**. The initial token may be the same, but when the model processes the complete sentence, its internal representation changes according to the context.

“ Contextualization in a Large Language Model (LLM) means giving the AI background information or clues so it understands what you mean, rather than just guessing based on a few isolated words”

There is biasness also that can happen. Suppose the model was trained with one-sided data, so that can also happen because the data itself was biased.

Note: Modern LLMs use Hybrid Search, which includes keyword and embedding search for better efficiency.

These embeddings can be used in recommendation, clustering, multimodal classification, duplicate detection and RAG also. We will remember these things and check them once we learn about them in more detail in the future.

Just keep in mind, an embedding is a general technique for representing information numerically.

Misconception

- ❑ An embedding is a dictionary of tokens
 - ❑ An embedding is a numerical representation capturing learned relationships.
 - ❑ Each dimension has one clear human meaning
 - ❑ Meaning is distributed across many dimensions.
 - ❑ Similar embeddings mean identical meaning
 - ❑ They may indicate topic similarity, association, opposition, category membership, or shared context
 - ❑ Embedding similarity proves a statement is true
 - ❑ Similarity measures relatedness, not truth.
 - ❑ One embedding model works equally well for every task
 - ❑ Performance depends on: Language, Domain, Data type, Text length, Training objective, etc
 - ❑ Visual clusters perfectly represent the original space
 - ❑ Two-dimensional and three-dimensional projections lose information.
 - ❑ A larger embedding dimension is always better
 - ❑ Larger representations create trade-offs in storage, latency, cost, and quality
- That's all for this episode. We will go more deep in the next one. Thanks, keep learning! ❑

Continue Exploring

COLLECTION

Best Web Performance tools

Continue exploring notes from the digital garden.

COLLECTION

Web Socket Lets Chat

Continue exploring notes from the digital garden.



Curated by

Ashutosh Anand Tiwari

Planting free, open-source tech notes and engineering guides so developers worldwide can learn, revise, and excel without paywalls.

Follow on Instagram

Star on GitHub ★

QUICK NAVIGATION

- [Course Notes](#)
- [Products](#)
- [Join Namaste Dev Club](#)
- [Tech Blogs](#)
- [Support Garden](#)

CONNECT & SOCIALS

X

Instagram

Globe

Headphones

Calendar

P

YouTube

Phone

